



Technical Document

Manage Backlog — SDD

SOFTWARE DESIGN DOCUMENT

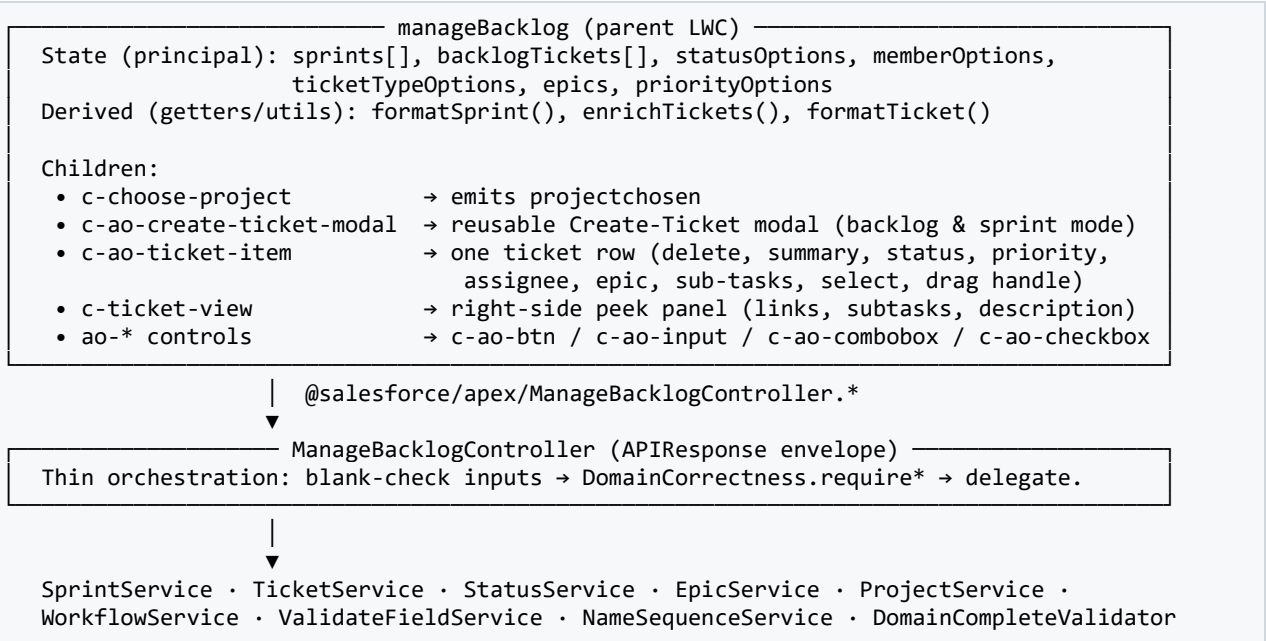
Manage Backlog · Sprint Planning Board

Software Design Document (SDD) — Manage Backlog

Feature: Manage Backlog (Sprint planning board) **Apex controller:** force-app/main/default/classes/controller/managebacklog/ManageBacklogController.cls **LWC:** force-app/main/default/lwc/manageBacklog/ **Date:** 2026-06-20

Image references below follow the requested marker convention: the image file name is wrapped between `#####` lines, immediately before the embedded image. All images live in `force-app/backlog-images/`.

1. Architecture Overview



Response envelope. Every controller method returns APIResponse { success, message, data }. The LWC treats success === false (or a thrown Apex error) as failure and toasts the message.

Data shape used by the page (from loadBacklogData): { sprints, status, ticketTypes, members, epics, backlogTickets, priorityOptions }.

2. API Calls (Apex @AuraEnabled methods)

Legend: **C** = cacheable=true (wire-friendly), **I** = imperative (DML).

2.1 Page bootstrap & search

Method	Kind	Params	Returns (`data`)
loadBacklogData	C	projectId	{sprints,status,ticketTypes,members,epics,backlogTicket

loadBacklogTickets	C	projectId, offset, pageSize	Ticket__c[]
loadTicketsBySprint	C	sprintId, offset, pageSize	Ticket__c[]
loadTicketBySearchTerm	C	projectId, searchTerm	Ticket__c[] (SOSL)
loadTicketTypes	C	projectId	TicketType__c[]
loadMembers	C	projectId	ProjectMember__c[]
getProjectStatuses	C	projectId	Status__c[]
loadUsers	C	—	User[]
loadProjects	C	—	Project__c[]

2.2 Sprint lifecycle

Method	Kind	Params	Effect
createSprint		duration, startDate, goal, projectId	insert sprint (future, totals 0, name auto)
updateSprint		sprintId, duration, startDate, goal	update sprint fields
startSprint		sprintId	future → in_progress (one active per project)
completeSprint		sprintId	in_progress → completed (drops from page)
deleteSprint		sprintId	soft delete + clear Sprint__c on its tickets

2.3 Ticket create / update / move / delete

Method	Kind	Params	Effect
createTicketFromBacklog		summary, ticketTypeId, currentStateId, description, storyPoint, assignedToId, epicId, priority	create sprint-less ticket

createTicketFromSprint		...same... + sprintId	create ticket in sprint + bump totals
updateTicket		ticketId, summary, description, storyPoint, priority, assignedToId, currentStateId	patch fields
updateTicketSummary		ticketId, summary	inline summary edit
updateTicketDescription		ticketId, description	description edit
updateTicketPriority		ticketId, priority	priority combo (✓)
assignTicket		ticketId, memberId	assignee combo
changeTicketState		ticketId, fromStatusId, toStatusId	workflow transition; returns {isEndStatus, updatedSprint}
updateTicketEpic		ticketId, epicId	set/clear epic
moveTicketToSprint		ticketId, sprintId	link + increase sprint totals; returns {updatedSprint}
moveTicketToBacklog		ticketId	unlink + decrease totals; returns {updatedSprint}
moveTicketPosition		movedTicketId, beforeTicketId	reorder (recompute Score__c); returns {movedTicket}
deleteTicket		ticketId	soft delete; returns {updatedSprint}
deleteTickets		ticketIds[]	bulk soft delete; returns {updatedSprints}
linkToTicket		fromTicketId, toTicketId, linkType	create ticket link; returns {ticketLink}
loadTicketLinkedTo	C	ticketId	{ticketLinkTo[]}
loadTicketLinkedToType	C	—	link-type options

2.4 Epic

Method	Kind	Params
createEpic		name, summary, projectId, description, startDate, endDate

updateEpic	I	epicId, name, summary, description, startDate, endDate
getEpicsByProject	C	projectId
getEpicById	C	epicId

2.5 Sub-task

Method	Kind	Params
loadSubtasks	C	ticketId
createSubtask	I	summary, ticketId, description, assigneeId, currentStateId, storyPoint, startDate
updateSubtaskSummary	I	subtaskId, summary
updateSubtask	I	subtaskId, summary, description, assigneeId, currentStateId, storyPoint
assignSubtask	I	subtaskId, memberId
deleteSubtask	I	subtaskId
deleteSubtasks	I	subtaskIds[]

2.6 Wires actually bound in manageBacklog.js

- @wire(loadTicketLinkedToType) → _ticketLinkedToTypeOptions
- @wire(loadTicketBySearchTerm, { projectId: '\$_projectId', searchTerm: '\$_ticketViewSearchTerm' })
- @wire(loadTicketLinkedTo, { ticketId: '\$_linkedToTargetTicketId' })
- @wire(loadSubtasks, { ticketId: '\$_subtasksTargetTicketId' })

All other reads/writes are **imperative** Apex calls inside handlers (then/catch/finally), flipping isLoading and toasting via _showSuccess / _showError.

3. UI / UX Specification

3.1 Page load — Backlog + non-completed sprint containers

On load the page renders the **Sprints** panel (every future/in_progress sprint container, collapsed) and the **Backlog** panel beneath it. Each sprint header carries the name, the StartDate → endDate range, the status badge (e.g. IN_PROGRESS), and the action buttons **Start / Complete / Edit / Delete / + Ticket**. The backlog header carries **+ Sprint** and **+ Ticket**.

```
#####
load-page-state-v00.png
#####
```



load-page-state-v00.png
 #####

3.2 Sprint container — dates, status, buttons, story-point line, paged tickets

Expanding a sprint shows the **goal** line, the **Story Points** progress line (ended / total SP · percent% with a progress bar), the paged list of ticket containers (5 per page, **Page N** with Prev/Next chevrons). This is the container that holds *many* ticket containers and the total-vs-ended story-point indicator.

expand-sprint-state-v01.png
 #####



- **Start** → confirm → startSprint → status future → in_progress.
- **Complete** → confirm → completeSprint → status in_progress → completed; the sprint ****no longer**

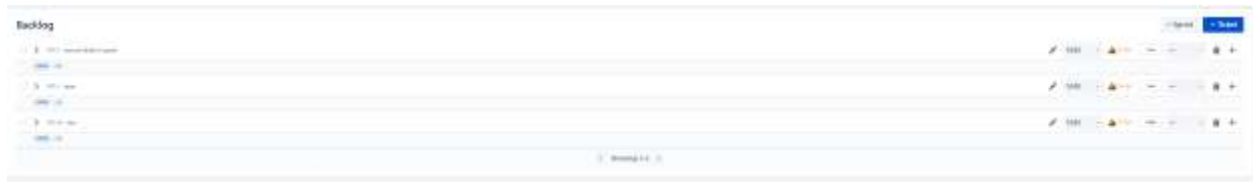
shows****** on the page.

- **Edit** → opens the Edit-Sprint modal pre-filled with the current values.
- **Delete** → confirmation pop-up; on confirm deleteSprint (tickets returned to backlog).
- **+ Ticket** → opens the reusable Create-Ticket modal in *sprint* mode.

3.3 Backlog container — paged ticket rows

The backlog lists active, sprint-less tickets ordered by score, 5 per page, with a **Showing 1–N** indicator and Prev/Next chevrons. Tickets can be dragged from here onto a sprint.

backlog-container.png
#####



3.4 Create-Sprint modal (+ Sprint)

Fields: **Duration (days) ***, **Start Date ***, **Goal ***. Buttons **Cancel** / **Create**. Submitting calls `createSprint(duration, startDate, goal, projectId)`.

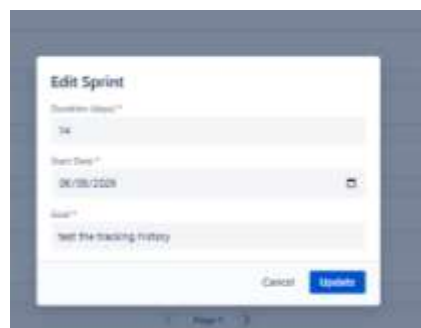
create-sprint-modal.png
#####



3.5 Edit-Sprint modal (Edit)

Same fields as create, **pre-filled** with the sprint's current Duration / Start Date / Goal. Button label is **Update**; submitting calls `updateSprint(sprintId, duration, startDate, goal)`.

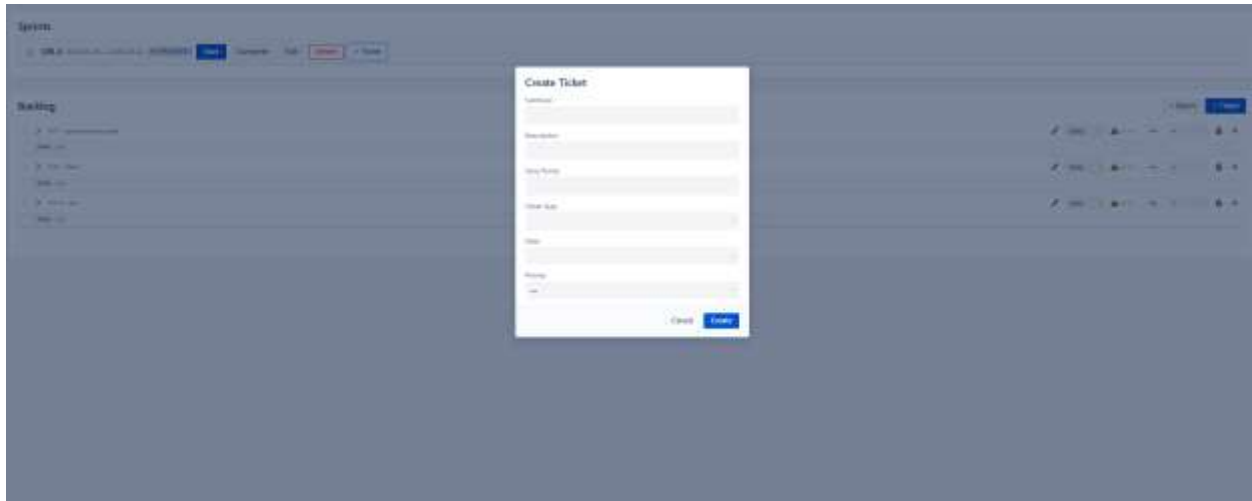
edit-sprint-modal.png
#####



3.6 Create-Ticket modal (+ Ticket — reused across the project)

The single `c-ao-create-ticket-modal` is reused for both backlog and sprint creation. Fields: **Summary, Description, Story Points, Ticket Type, State, Priority**. Buttons **Cancel / Create**. In sprint mode the modal receives `sprint-id` and the parent calls `createTicketFromSprint`; otherwise `createTicketFromBacklog`.

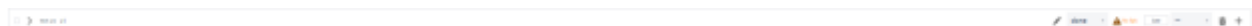
```
#####
create-ticket-modal.png
#####
```



3.7 Ticket container (row)

A row shows the ticket **name**, **summary**, **epic** label (**No Epic** when unset, clickable), **current status** combo, **priority** combo, **assignee** combo, plus the type/story-point chip (e.g. `STORY 3 SP`). Trailing controls: edit-summary (pencil), epic label, status combo, priority combo, assignee combo, delete (trash), and add-subtask (+). The leading chevron expands the sub-tasks; the leading checkbox selects for bulk delete.

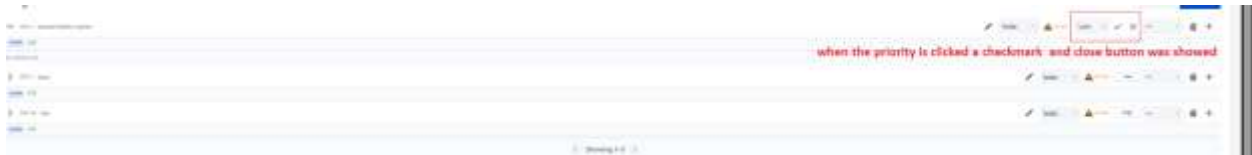
```
#####
ticket-container-row.png
#####
```



3.8 Change priority — combo with confirm (✓) and close (✕)

Clicking the priority combo reveals a **checkmark** (confirm) and a **close** button. Confirming validates the value (Critical / High / Medium / Low) and calls `updateTicketPriority`.

```
#####
change-priority-value.png
#####
```



3.9 Expand ticket — sub-tasks list

The ticket's expand chevron lazy-loads (loadSubtasks) and lists its sub-tasks beneath the row. Each sub-task supports summary edit, assignee change, and delete.

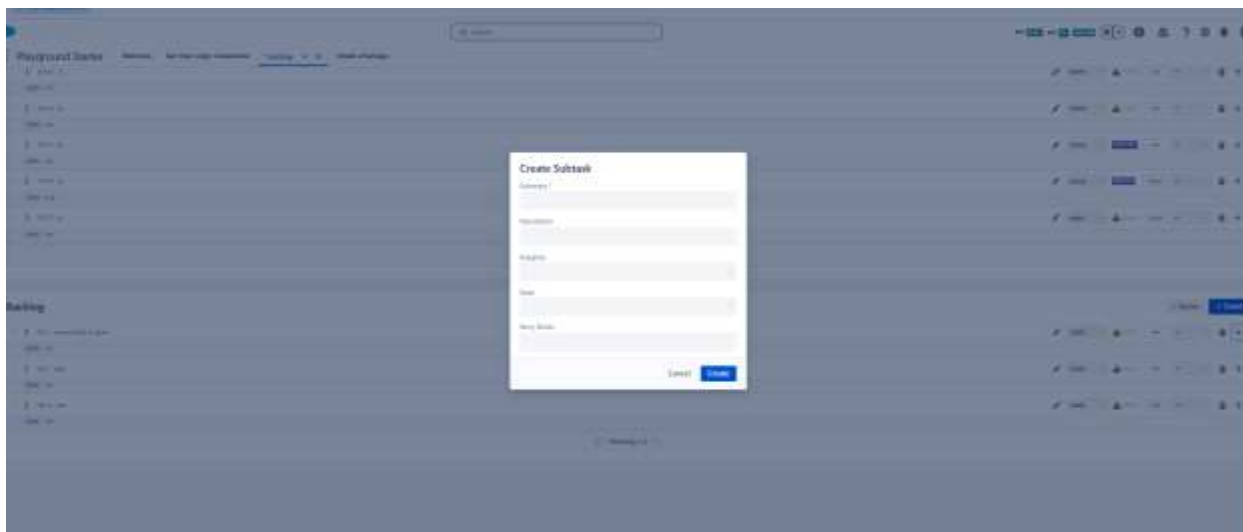
show-sub-task-for-ticket-container.png
 #####



3.10 Create-Subtask modal (+ on a ticket)

Fields: **Summary ***, **Description**, **Assignee**, **State**, **Story Points**. Buttons **Cancel / Create**; submitting calls createSubtask.

create-sub-task-modal.png
 #####



3.11 Update Epic Parent modal (epic label → choose existing or create new)

Clicking the epic label opens the epic modal. The user can pick an existing epic or **create a new one**. The create form fields: **Epic Name ***, **Summary ***, **Description**, **Start Date**, **End Date**. Creating calls createEpic then updateTicketEpic; choosing an existing epic calls updateTicketEpic directly.

create-epic-modal.png
 #####

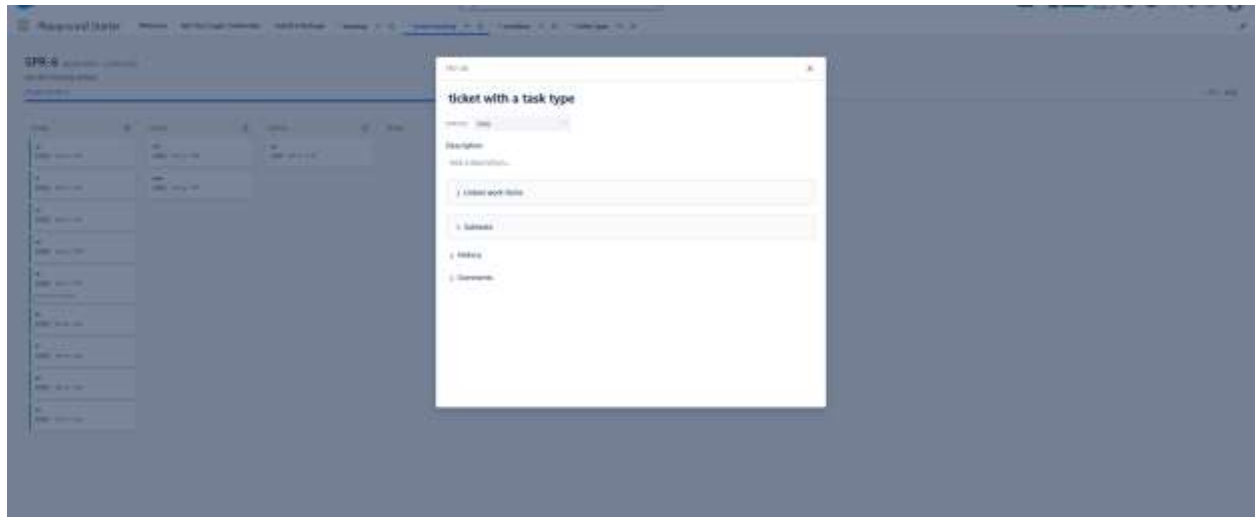
3.12 Ticket view panel (c-ticket-view)

Clicking a ticket row dispatches `ticketviewopen` and opens the right-side `c-ticket-view` panel for that ticket (the parent keeps `_activeTicketViewId` and feeds the panel via `_findTicketById`). The header shows the ticket Name and a Close (×). Body sections:

Summary — click the title to edit inline (`c-ao-input`) with confirm (✓) / cancel (×), saving via `ticketsummaryupdate` → `updateTicketSummary`. **Status** — a combo box bound to `statusOptions` that fires `ticketstatuschange` → `changeTicketState`.

Description — rendered with `lightning-formatted-rich-text`; clicking it (or the “Add a description...” placeholder when empty) swaps in a `lightning-input-rich-text` editor with Save / Cancel (`ticketdescriptionupdate` → `updateTicketDescription`). **Linked work items** — a collapsible section; expanding fires `ticketlinkedtoexpand` (`@wire loadTicketLinkedTo`) and lists the items under the blocks label; the + reveals an add-link form with a Link-type combo (`loadTicketLinkedToType`) and a ticket auto-complete that searches on ≥ 2 characters (`ticketsearch` → `@wire loadTicketBySearchTerm`), with Cancel / Link — Link is enabled only once both a type and a target ticket are chosen (`linkToTicket`). **Subtasks** — a collapsible section; expanding fires `subtasksexpand` (`@wire loadSubtasks`) and lists each sub-task (name, summary, assignee); the + reveals an inline create form (Summary *, Assignee, Current State, Start Date, Story Point, Description) with Cancel / Create (`subtaskcreate` → `createSubtask`).

Ticket-view-panel.png
#####



Ticket-view-status-combo.png
#####



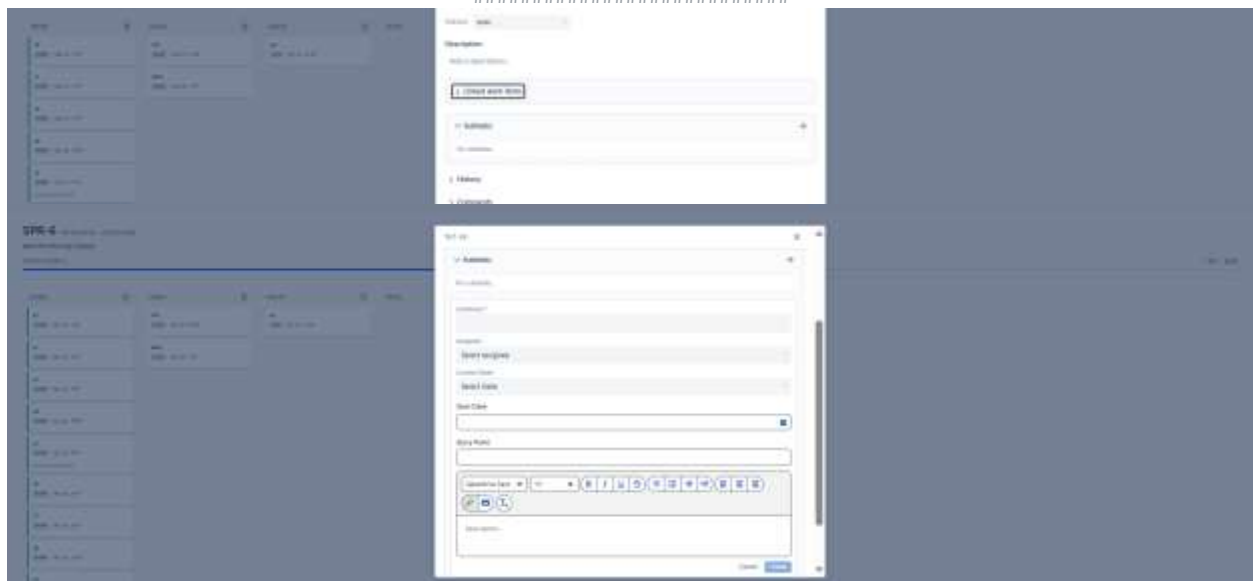
Ticket-view-description-editor.png
#####



Ticket-view-linked-work-items.png
#####



Ticket-view-subtask-form.png
#####



4. Traceability (SRS → API)

SRS	API call(s)
FR-2 page load	loadBacklogData
FR-4 / FR-8 paging	loadBacklogTickets, loadTicketsBySprint
FR-9..FR-13 sprint actions	startSprint, completeSprint, updateSprint, deleteSprint
FR-14/FR-15 create ticket	createTicketFromBacklog, createTicketFromSprint
FR-17 inline ops	updateTicketSummary, changeTicketState, updateTicketPriority, assignTicket, updateTicketEpic, deleteTicket
FR-18 epic	createEpic, updateTicketEpic, getEpicsByProject

FR-19 bulk delete	deleteTickets
FR-20/FR-21 sub-tasks	createSubtask, loadSubtasks, updateSubtaskSummary, assignSubtask, deleteSubtask, deleteSubtasks
FR-22..FR-24 move/reorder	moveTicketToSprint, moveTicketToBacklog, moveTicketPosition
FR-25 peek panel links	loadTicketLinkedTo, loadTicketLinkedToType, linkToTicket, loadTicketBySearchTerm
FR-26..FR-31 ticket view panel	updateTicketSummary, changeTicketState, updateTicketDescription, loadTicketLinkedTo, loadTicketBySearchTerm, linkToTicket, loadSubtasks, createSubtask